**KissanDost AI**

**Product Requirements Document (PRD)**

**Version:** 1.0

**Document Status:** Draft

**Product Type:** AI Agricultural Decision Support System (AI-ADSS)

**Prepared By:** BroMinds

**Last Updated:** November 2026

---

## Chapter 1 — Executive Summary

---

### 1.1 Executive Overview

Agriculture remains one of the most important sectors in Pakistan and is the primary source of income for millions of families in Khyber Pakhtunkhwa (KP). Despite advances in digital technologies, many smallholder farmers still rely on informal advice from pesticide dealers, neighboring farmers, or personal experience when making crop treatment decisions. These decisions directly affect crop health, production costs, and overall yield.

While AI-powered crop disease detection tools have become increasingly available, most stop after identifying a disease, leaving farmers to determine the next course of action on their own. This gap often results in delayed interventions, inappropriate pesticide use, unnecessary expenses, and avoidable crop losses.

**KissanDost AI** addresses this gap by evolving beyond disease detection into an **AI-powered Agricultural Decision Support System (ADSS)**. Rather than simply identifying plant diseases, the platform provides localized, explainable, and actionable recommendations based on multiple contextual factors, including crop condition, disease severity, weather forecasts, crop growth stage, trusted agricultural knowledge, and historical farm data.

The platform also introduces **Community Disease Intelligence**, an anonymized disease surveillance capability that identifies emerging disease trends across farming communities. This enables both farmers and agricultural authorities to respond proactively to potential outbreaks.

KissanDost AI is designed to be simple enough for farmers to use while incorporating advanced AI techniques behind the scenes. The user experience is intentionally

minimal: farmers capture a photo of an affected crop, and the platform transforms that input into a clear recommendation delivered in their preferred language.

---

### 1.2 Product Overview

KissanDost AI is an AI-enabled web platform that supports farmers throughout the crop treatment decision process.

Instead of asking farmers to interpret disease names, scientific reports, or pesticide labels, the platform provides a clear recommendation that answers practical questions:

- What is affecting my crop?
- How serious is the problem?
- Should I act now or monitor the situation?
- When is the best time to spray?
- Which treatment is appropriate?
- Why is this recommendation being made?
- When should I inspect the crop again?

The recommendation is generated using a combination of:

- Computer Vision
- Retrieval-Augmented Generation (RAG)
- Rule-Based Decision Engine
- Weather Intelligence
- Farm History
- Explainable AI

---

### 1.3 Vision Statement

**To empower every farmer in Pakistan with trusted, explainable, and localized AI-powered agricultural decision support, improving productivity, sustainability, and food security.**

---

### 1.4 Mission Statement

KissanDost AI aims to reduce uncertainty in agricultural decision-making by providing accessible, practical, and evidence-based recommendations through artificial intelligence.

The platform seeks to bridge the gap between farmers and agricultural expertise by making trusted guidance available directly from a smartphone or computer.

---

### 1.5 Elevator Pitch

**KissanDost AI is an AI-powered Agricultural Decision Support System that transforms a simple crop photo into an explainable treatment plan by combining computer vision, official agricultural knowledge, weather intelligence, and farm history. It also provides anonymized community disease monitoring to support early outbreak detection across Khyber Pakhtunkhwa.**

---

### 1.6 Problem Statement

Smallholder farmers frequently make important crop treatment decisions without timely access to agricultural experts.

As a result, many depend on informal advice from pesticide dealers, neighboring farmers, or social media communities. While such advice may occasionally be helpful, it is often inconsistent, difficult to verify, and not tailored to the farmer's specific field conditions.

Current digital tools generally focus on disease detection rather than decision support. They identify a disease but do not provide sufficient context regarding treatment timing, severity, weather conditions, or follow-up actions.

This creates uncertainty, increases production costs, and reduces agricultural productivity.

---

### 1.7 Proposed Solution

KissanDost AI transforms disease identification into complete agricultural decision support.

Using artificial intelligence, the platform:

- Identifies crop diseases from photographs.
- Estimates disease severity.
- Retrieves trusted agricultural recommendations.

- Considers local weather conditions.
- Determines appropriate treatment timing.
- Verifies treatment suitability.
- Explains every recommendation.
- Monitors disease trends across communities.

The result is a simple recommendation that helps farmers make informed decisions with confidence.

---

**1.8 Product Goals**

The primary goals of KissanDost AI are:

1. Reduce unnecessary pesticide use.
2. Improve treatment timing.
3. Increase crop productivity.
4. Reduce financial losses.
5. Improve access to agricultural expertise.
6. Provide explainable AI recommendations.
7. Support government disease monitoring.
8. Enable early outbreak detection.

---

**1.9 Success Metrics (KPIs)**

| Objective | KPI |
|---|---|
| Improve decision quality | ≥80% of pilot users report increased confidence |
| Reduce unnecessary pesticide use | Target 15–20% reduction during pilot |
| Faster diagnosis | Recommendation generated in under 10 seconds |
| User adoption | 500+ pilot users |
| Recommendation quality | ≥85% user satisfaction |
| System uptime | ≥99% during pilot |

| Objective | KPI |
| --- | --- |
| Community surveillance | Disease hotspot alerts generated successfully |

---

### 1.10 Scope of MVP

The MVP focuses on delivering the minimum set of features required to validate the product concept.

Included:

- User authentication
- Crop image upload
- Disease detection
- Severity estimation
- Weather-aware recommendations
- RAG-based agricultural guidance
- Explainable recommendations
- Farm history
- Pashto/Urdu support
- Community Disease Intelligence

Excluded from MVP:

- Drone integration
- IoT sensor integration
- Satellite imagery
- Marketplace
- Online pesticide purchasing
- Yield prediction
- Offline inference
- Livestock support

---

### 1.11 Why Now?

Several trends make this solution particularly relevant:

- Growing smartphone adoption among farmers.
- Increased availability of AI APIs and computer vision models.
- Public access to agricultural knowledge resources.
- Need for climate-resilient farming.
- Government interest in digital agriculture.
- Increasing frequency of crop disease outbreaks.

These factors make it practical to deliver AI-assisted agricultural decision support at scale.

---

### 1.12 Key Differentiator

Unlike traditional crop disease detection applications, KissanDost AI does not stop after identifying a disease.

It continues through the complete decision-making process by combining image analysis, contextual information, trusted agricultural knowledge, and explainable recommendations into a single, easy-to-understand action plan.

The addition of Community Disease Intelligence further extends the platform from an individual farmer's assistant to a tool that can support district-level agricultural monitoring and early warning.

---

## Chapter 2 — Vision, Mission & Objectives

---

### 2.1 Product Vision

KissanDost AI envisions a future where every farmer, regardless of location or technical expertise, has access to timely, trustworthy, and localized agricultural guidance.

The platform aims to become the first choice for crop health decisions in Pakistan by combining AI with official agricultural knowledge in a way that is simple, transparent, and practical.

---

### 2.2 Mission

Our mission is to make expert agricultural decision-making accessible to every farmer through explainable artificial intelligence that is affordable, reliable, and easy to use.

---

### 2.3 Long-Term Vision

Over the next five years, KissanDost AI aims to evolve from a crop treatment assistant into a comprehensive digital agriculture platform supporting:

- Crop disease management
- Pest monitoring
- Fertilizer recommendations
- Irrigation guidance
- Yield forecasting
- Climate adaptation
- Government advisory services
- Agricultural analytics

---

### 2.4 Core Product Principles

**Simplicity First**

Farmers should receive one clear recommendation rather than complex technical reports.

**Explainability**

Every recommendation must include the reasoning behind it.

**Trust**

Recommendations should be grounded in trusted agricultural sources rather than unsupported AI-generated advice.

**Localization**

Support local crops, languages, weather conditions, and farming practices.

**Accessibility**

The platform should remain usable by farmers with limited digital literacy.

**Human-Centered AI**

The system supports decision-making but does not replace agricultural experts.

---

**2.5 Business Objectives**

- Build a scalable AI platform for agriculture.
- Demonstrate practical AI adoption in farming.
- Establish partnerships with agricultural institutions.
- Enable future expansion across Pakistan.
- Develop a sustainable technology platform.

---

**2.6 Farmer Objectives**

The platform should enable farmers to:

- Detect problems earlier.
- Reduce unnecessary costs.
- Improve crop health.
- Increase confidence in decisions.
- Access trusted guidance.
- Receive information in their preferred language.

---

**2.7 Government Objectives**

Support agricultural authorities by:

- Identifying disease hotspots.
- Monitoring regional outbreaks.
- Improving resource allocation.
- Supporting extension officers.
- Providing anonymized agricultural insights.

---

**2.8 North Star Metric**

**Percentage of farming decisions positively influenced by KissanDost AI recommendations.**

This metric reflects the core value of the platform: improving real-world decisions rather than simply increasing app usage.

---

### 2.9 Product Success Criteria

KissanDost AI will be considered successful if it:

- Helps farmers make faster and more informed treatment decisions.
- Demonstrates measurable reductions in unnecessary pesticide use during pilot testing.
- Provides recommendations that users find understandable and trustworthy.
- Supports agricultural authorities with meaningful community disease insights.
- Establishes a scalable foundation for future digital agriculture services.

## Chapter 3 — Problem Analysis

---

### 3.1 Introduction

Agriculture contributes significantly to Pakistan's economy and remains the primary livelihood for millions of people. In Khyber Pakhtunkhwa (KP), a large percentage of farmers are smallholders who manage limited land with constrained financial and technical resources.

Crop diseases, pest infestations, changing weather patterns, and delayed interventions significantly reduce productivity every year. While agricultural research and advisory services exist, they often do not reach farmers quickly enough when decisions need to be made.

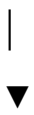
The problem is therefore **not a lack of agricultural knowledge**, but **limited access to the right knowledge at the right time.**

---

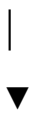
### 3.2 Current Farming Workflow

Today, a typical farmer follows this process:

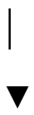
Crop shows unusual symptoms



Farmer notices damaged leaves



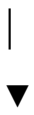
Asks neighbors or pesticide dealer



Receives different opinions



Purchases pesticide



Sprays crop



Waits several days



Either crop improves or problem worsens

This process is:

- Reactive
- Based on assumptions
- Difficult to verify
- Time-consuming
- Expensive

---

### 3.3 Core Problem

The primary challenge is **decision uncertainty**.

Farmers ask questions such as:

- Is this disease serious?
- Should I spray today?
- Which pesticide is suitable?
- Is treatment even necessary?
- Should I wait?
- Is rain going to reduce effectiveness?
- Should I contact an agriculture officer?

Existing solutions rarely answer these questions together.

---

### 3.4 Root Cause Analysis

| Problem | Root Cause |
|---|---|
| Wrong pesticide selection | Reliance on informal advice |
| Delayed treatment | Lack of timely expert access |
| Unnecessary spraying | Poor disease severity assessment |
| Financial loss | Trial-and-error decisions |
| Low productivity | Incorrect management practices |
| Disease spread | No community-level monitoring |

---

### 3.5 Existing Solutions

Farmers currently rely on:

**Local pesticide dealers**

Pros:

- Easily available

Cons:

- May recommend expensive or unnecessary products
- Advice varies by individual

---

**Neighbor recommendations**

Pros:

- Trusted relationships

Cons:

- Experience may not match current conditions

---

**Facebook / WhatsApp groups**

Pros:

- Quick responses

Cons:

- Advice quality is inconsistent
- Difficult to verify

---

**Agriculture extension offices**

Pros:

- Expert guidance

Cons:

- Limited availability
- Travel and waiting time
- Not immediately accessible

---

**AI disease detection apps**

Pros:

- Identify diseases

Cons:

- Limited decision support
- Often generic recommendations
- Minimal localization

---

**3.6 Gap Analysis**

Current solutions answer:

**"What disease is this?"**

Farmers actually need:

**"What should I do now?"**

This gap is where KissanDost AI provides value.

---

**3.7 Why Existing AI Is Not Enough**

Large Language Models can answer agricultural questions.

However, they generally do not automatically combine:

- Image understanding
- Disease severity
- Local weather
- Crop growth stage
- Farm history
- Trusted agricultural references
- Community disease trends

into one localized recommendation.

KissanDost AI orchestrates these data sources into a structured decision-support workflow.

---

**3.8 Stakeholder Analysis**

| Stakeholder | Need |
|---|---|
| Farmer | Clear recommendation |
| Agriculture Officer | Early disease reports |
| Government | Regional disease trends |
| Researchers | Anonymous field observations |
| Local Community | Reduced crop losses |

---

### 3.9 Problem Validation

The problem was identified through:

- Observation of common farming practices
- Review of digital agriculture solutions
- Discussions with farmers and local agricultural challenges
- Analysis of AI limitations in practical farming workflows

Future validation activities:

- Farmer interviews
- Pilot deployment
- Agriculture department feedback
- Field usability testing

---

### 3.10 Opportunity Statement

There is an opportunity to create a platform that transforms disease identification into complete agricultural decision support while simultaneously generating anonymized disease intelligence that benefits entire farming communities.

---

### 3.11 Problem Statement (Final)

Smallholder farmers lack timely access to localized, trustworthy decision support after crop diseases appear, resulting in incorrect treatments, unnecessary expenses, reduced yields, and avoidable crop losses.

---

## Chapter 4 — User Research & Personas

---

### 4.1 Target Users

KissanDost AI is designed for multiple user groups.

Primary:

- Smallholder farmers

Secondary:

- Agriculture extension officers

Tertiary:

- Government departments
- Researchers

---

### 4.2 Primary Persona

**Name**

Muhammad Khan

---

**Age**

46

---

**Occupation**

Farmer

---

**Location**

Mardan, KP

---

**Education**

Secondary school

---

**Farm Size**

4 acres

---

**Crops**

- Wheat
- Maize
- Tomato

---

**Goals**

- Protect crops
- Reduce expenses
- Increase yield

---

**Challenges**

- Limited expert access
- Unsure about pesticides
- Weather uncertainty
- Difficulty identifying disease severity

---

**Technology Usage**

- Android smartphone
- WhatsApp
- YouTube

Limited English proficiency.

---

**Needs**

Simple recommendations.

Not scientific reports.

---

**4.3 Secondary Persona**

**Name**

Ayesha

Agricultural Extension Officer

Goals:

- Assist more farmers
- Identify disease trends
- Monitor outbreaks
- Prioritize field visits

Needs:

- Dashboard
- Alerts
- Heat maps
- Reliable field reports

---

**4.4 Government Persona**

KP Agriculture Department

Needs:

- District statistics
- Disease hotspots
- Historical trends
- Early warning system
- Resource allocation

---

**4.5 Jobs-To-Be-Done (JTBD)**

When my crop develops unusual symptoms,

I want to quickly understand what action I should take,

so that I can protect my crop before the problem becomes worse.

---

**4.6 User Journey**

**Before**

Disease appears

↓

Confusion

↓

Ask dealer

↓

Buy pesticide

↓

Hope it works

---

**After**

Take Photo

↓

AI Analysis

↓

Localized Recommendation

↓

Best Spray Time

↓

Treatment Guidance

↓

Follow-up Reminder

---

**4.7 User Pain Points**

| Pain | Solution |
|---|---|
| Doesn't know disease | Vision AI |
| Unsure if serious | Severity estimation |
| Doesn't trust advice | Explainable recommendations |
| Doesn't know spray timing | Weather intelligence |
| Doesn't know pesticide | Treatment verification |
| Doesn't know what to do | Decision engine |

---

### 4.8 Accessibility Requirements

The application should support users with limited literacy by providing:

- Pashto
- Urdu
- English

Voice narration

Large buttons

Minimal typing

Simple navigation

High-contrast interface

Image-first interaction

---

### 4.9 User Goals

**Farmers**

- Save money
- Protect crops
- Receive trusted advice

---

**Government**

- Monitor diseases
- Improve response time
- Identify outbreaks

---

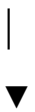
**Extension Officers**

- Reduce workload
- Prioritize visits
- Support more farmers

---

### 4.10 User Story Map

Open App



Capture Image



AI Analysis



Decision Recommendation



Treatment Guidance



Follow-up



Community Intelligence

---

### 4.11 Design Principles from User Research

The research informs several product principles:

**Simplicity Over Complexity**

Recommendations should be short and actionable.

---

**Explain Every Recommendation**

Users should understand *why* an action is suggested.

---

**Local Language First**

Recommendations should be available in Pashto and Urdu.

---

**Trust Before Automation**

The system should encourage consultation with agricultural experts for uncertain or severe cases rather than presenting itself as infallible.

---

**Minimize User Effort**

The entire workflow should require only:

- Open app
- Take photo
- Read or listen to recommendation

No complex forms.

No technical terminology.

---

### 4.12 User Research Summary

The research indicates that farmers are not seeking another AI chatbot or disease identifier. They need a practical assistant that reduces uncertainty and simplifies decision-making. By combining computer vision, trusted agricultural knowledge, weather awareness, and explainable recommendations, KissanDost AI is designed to fit naturally into the existing farming workflow while reducing costly errors.

## Chapter 5 — Market & Competitor Analysis

---

### 5.1 Market Overview

Agriculture remains one of Pakistan's largest economic sectors and employs a significant portion of the workforce. In Khyber Pakhtunkhwa, smallholder farmers cultivate crops such as wheat, maize, sugarcane, tobacco, rice, vegetables, and fruits. These farmers often face challenges including crop diseases, unpredictable weather, limited access to agricultural experts, and delayed interventions.

At the same time, smartphone usage, mobile internet availability, and AI services have become increasingly accessible. This creates an opportunity to deliver expert agricultural guidance directly to farmers without requiring them to travel or wait for field visits.

The opportunity is **not to replace agricultural experts**, but to extend their reach through AI-assisted decision support.

---

**5.2 Market Opportunity**

**Primary Market**

- Smallholder farmers in Khyber Pakhtunkhwa.
- Farmers in ARDC merged districts.
- Progressive farmers adopting digital tools.

---

**Secondary Market**

- Agriculture Extension Officers
- Agriculture Department
- NGOs working in agriculture
- Agricultural universities
- Research institutions

---

**Future Expansion**

- Punjab
- Sindh
- Balochistan
- National agriculture programs
- International adaptation for similar farming regions

---

**5.3 Current Alternatives**

Today farmers commonly use:

- Local pesticide dealers
- Neighbor recommendations
- Facebook groups
- WhatsApp groups
- Google Search
- YouTube videos
- General AI assistants
- Plant disease detection apps

Each solves only part of the problem.

---

**5.4 Competitor Analysis**

**Competitor 1 — Plantix**

**Strengths**

- Disease detection
- Large disease database
- Mobile application

**Weaknesses**

- Generic recommendations
- Limited localization
- No farm history
- No community outbreak intelligence
- Limited explainability

---

**Competitor 2 — PlantVillage**

**Strengths**

- Research-backed
- Good disease identification

**Weaknesses**

- Primarily focused on identification
- Less localized for Pakistan
- No integrated decision workflow

---

**Competitor 3 — Google Lens**

**Strengths**

- Easy to use
- Good image recognition

**Weaknesses**

- Not agriculture-specific
- No treatment planning
- No decision support

---

**Competitor 4 — ChatGPT / General LLMs**

**Strengths**

- Can answer agricultural questions
- Conversational
- Flexible

**Weaknesses**

- Requires users to ask the right questions
- May produce inconsistent recommendations without grounded data
- No farm memory
- No weather-aware workflow
- No structured agricultural decision engine
- No community disease intelligence

---

**Competitor 5 — Local Dealers**

**Strengths**

- Easily accessible
- Local knowledge

**Weaknesses**

- Advice quality varies
- Commercial bias may exist
- No standardized guidance

---

### 5.5 Competitive Comparison

| Feature | KissanDost AI | Plantix | ChatGPT | Google Lens |
|---|---|---|---|---|
| Disease Detection | ✅ | ✅ | ⚠️ | ⚠️ |
| Severity Estimation | ✅ | Limited | ⚠️ | ❌ |
| Weather-aware Advice | ✅ | ❌ | ❌ | ❌ |
| RAG-based Recommendations | ✅ | ❌ | Limited | ❌ |
| Explainable Decisions | ✅ | Limited | Limited | ❌ |
| Farm History | ✅ | ❌ | ❌ | ❌ |
| Product Verification | ✅ | ❌ | ❌ | ❌ |
| Community Intelligence | ✅ | ❌ | ❌ | ❌ |
| Government Dashboard | ✅ | ❌ | ❌ | ❌ |
| Pashto Support | ✅ | Limited | Depends | ❌ |

---

### 5.6 SWOT Analysis

| Strengths | Weaknesses |
|---|---|
| AI-powered decision support | Dependent on image quality |
| Local language support | Internet required for MVP |
| Explainable recommendations | Initial knowledge base size |

| Strengths | Weaknesses |
| --- | --- |
| Community intelligence | Limited pilot data |

---

| Opportunities | Threats |
| --- | --- |
| Digital agriculture growth | User trust in AI |
| Government partnerships | Similar future products |
| Expansion to Pakistan | Connectivity issues |
| Climate adaptation | Regulatory changes |

---

**5.7 Competitive Advantage**

Our competitive advantage is **context-aware decision support**.

Most products stop at:

Disease Identification

KissanDost AI continues with:

- Disease severity
- Weather conditions
- Growth stage
- Trusted agricultural knowledge
- Treatment verification
- Explainable recommendations
- Follow-up monitoring
- Community intelligence

The product therefore solves the complete decision workflow instead of a single task.

---

**5.8 Product Positioning**

KissanDost AI is positioned as an **AI Agricultural Decision Support Platform**, not as a crop disease detector.

This distinction is important because the platform supports the farmer throughout the decision-making process rather than simply recognizing symptoms.

---

### 5.9 Unique Selling Proposition (USP)

**"One photo. One trusted decision."**

Instead of overwhelming farmers with information, KissanDost AI provides a single, explainable recommendation tailored to their local conditions.

---

### 5.10 Strategic Differentiators

1. Explainable AI recommendations
2. Weather-aware treatment planning
3. RAG using trusted agricultural knowledge
4. Treatment verification
5. Community Disease Intelligence
6. Government dashboard
7. Pashto-first experience

---

## Chapter 6 — Product Requirements

---

### 6.1 Product Goal

Develop an AI-powered Agricultural Decision Support System that enables farmers to make informed crop treatment decisions using image analysis, contextual information, and trusted agricultural guidance.

---

### 6.2 Product Scope

**In Scope (MVP)**

- User Registration & Login
- Farmer Profile
- Farm Registration

- Crop Scan
- Disease Detection
- Severity Estimation
- Weather Integration
- AI Recommendation
- Explainable Recommendation
- Voice Output
- Scan History
- Community Disease Intelligence
- Government Dashboard (basic analytics)

---

**Out of Scope (MVP)**

- Drone imagery
- IoT sensor integration
- Automatic irrigation control
- Marketplace
- Online ordering
- Fertilizer optimization
- Yield prediction
- Offline AI inference

---

**6.3 Product Objectives**

**Objective 1**

Reduce uncertainty in crop treatment decisions.

---

**Objective 2**

Improve accessibility to agricultural guidance.

---

**Objective 3**

Promote responsible pesticide use.

---

**Objective 4**

Support early disease monitoring.

---

**Objective 5**

Provide trustworthy AI recommendations.

---

### 6.4 Success Metrics

| Metric | Target |
|---|---|
| Average recommendation time | <10 seconds |
| Scan success rate | >95% |
| AI confidence visibility | 100% |
| User satisfaction | >85% |
| System availability | >99% |
| Recommendation explanation | 100% |

---

### 6.5 Assumptions

The MVP assumes:

- Farmers possess Android smartphones or access to a shared device.
- Internet connectivity is available during image upload.
- Public agricultural guidance documents are accessible for the knowledge base.
- Weather APIs provide reliable forecasts.

---

### 6.6 Constraints

**Technical**

- Limited hackathon time.
- API usage limits.
- Budget limitations.

---

**Product**

- MVP only.
- Focus on selected crops.
- Web-first implementation.

---

**AI**

- Existing foundation models only.
- No custom model training.

---

**6.7 Product Principles**

Every feature should satisfy these principles:

**Simplicity**

Minimal user interaction.

---

**Explainability**

Every recommendation must explain *why*.

---

**Reliability**

Ground recommendations using trusted agricultural knowledge.

---

**Localization**

Support local languages and local agricultural practices.

---

**Privacy**

User data belongs to the farmer.

---

**6.8 Core Modules**

**Module 1**

Authentication

Purpose:

Secure user accounts.

---

**Module 2**

Farm Management

Store farms, crops, and history.

---

**Module 3**

Crop Scan

Image upload and preprocessing.

---

**Module 4**

Vision AI

Disease detection.

---

**Module 5**

Decision Engine

Generate recommendations.

---

**Module 6**

Knowledge Engine

Retrieve trusted agricultural information using RAG.

---

**Module 7**

Weather Intelligence

Determine optimal treatment timing.

---

**Module 8**

Product Verification

Validate pesticide suitability.

---

**Module 9**

Farm Timeline

Historical tracking.

---

**Module 10**

Community Disease Intelligence

Aggregate anonymized reports for hotspot detection.

---

**Module 11**

Government Dashboard

Visualize disease trends and alerts.

---

**6.9 Product Requirements Matrix**

| Requirement | Priority |
|---|---|
| Authentication | High |
| Crop Scan | High |
| Disease Detection | High |
| AI Recommendation | High |
| Weather Integration | High |

| Requirement | Priority |
|---|---|
| RAG | High |
| Explainability | High |
| Voice Output | Medium |
| Scan History | Medium |
| Product Verification | Medium |
| Community Intelligence | Medium |
| Government Dashboard | Low (Hackathon MVP) |

---

### 6.10 Definition of MVP Success

The MVP will be considered successful if a farmer can:

1. Register an account.
2. Register a farm.
3. Capture or upload a crop image.
4. Receive an AI-generated recommendation within 10 seconds.
5. Understand why the recommendation was made.
6. View previous scans.
7. Receive weather-aware treatment advice.
8. Contribute anonymized disease data (with consent) for community monitoring.

---

### 6.11 Product Requirements Summary

The MVP intentionally prioritizes **quality over quantity**. Rather than building many disconnected features, KissanDost AI focuses on delivering a seamless end-to-end decision-support workflow that farmers can trust and use repeatedly. Every feature included in the MVP directly contributes to the platform's core promise: helping farmers make better crop treatment decisions through explainable, AI-assisted guidance.

---

### 📌 Notes for the Final PRD

As we continue, I recommend adding:

- Architecture diagrams
- ER diagrams
- Sequence diagrams
- User flow diagrams
- RAG pipeline diagrams
- Decision engine flowcharts

These visual elements will elevate the final document from a good PRD to one that feels like a professional product and engineering specification.

## Chapter 7 — Functional Requirements

---

### 7.1 Introduction

This chapter defines the functional behavior of KissanDost AI. Each requirement includes its purpose, priority, user flow, business rules, acceptance criteria, and future enhancements.

**Priority Legend**

| Priority | Meaning |
|---|---|
| P0 | Critical for MVP |
| P1 | High Priority |
| P2 | Medium Priority |
| P3 | Future Enhancement |

---

### FR-001 User Registration & Authentication

**Priority:** P0

**Description**

Users shall be able to create an account using email or Google Sign-In.

**Inputs**

- Full Name
- Email
- Password

**Outputs**

- Secure account
- Auth token
- User profile

**Business Rules**

- Email must be unique.
- Password must meet minimum security requirements.
- JWT-based authentication.

**Acceptance Criteria**

- User can register successfully.
- Duplicate emails are rejected.
- Login persists across sessions.
- User can log out securely.

**Future**

- Phone number login
- OTP authentication

---

**FR-002 Farm Management**

**Priority:** P0

**Description**

Users can register one or more farms.

**Data Captured**

- Farm Name
- District
- Village

- Crop Type
- Area
- Planting Date

**Business Rules**

- One user may own multiple farms.
- Farm location is optional for MVP but recommended.

**Acceptance Criteria**

- User can create, edit, and delete farms.
- Farm history remains linked to each farm.

---

**FR-003 Crop Scan**

**Priority:** P0

**Description**

Users upload or capture a crop image for analysis.

**Inputs**

- Camera
- Gallery

**Validation**

- JPG/PNG
- Maximum size: 10 MB
- Minimum resolution: 512 × 512 px

**Acceptance Criteria**

- Image uploads successfully.
- Upload errors are handled gracefully.
- Preview displayed before analysis.

---

**FR-004 Disease Detection**

**Priority:** P0

**Description**

AI identifies crop and disease.

**Outputs**

- Crop Name
- Disease Name
- Confidence Score

**Business Rules**

- If confidence < threshold, display uncertainty message.
- Unknown cases are flagged for manual review.

**Acceptance Criteria**

- Detection completes within target response time.
- Confidence is displayed.
- Unknown predictions handled safely.

---

**FR-005 Severity Assessment**

**Priority:** P1

**Description**

Estimate disease severity.

**Severity Levels**

- Healthy
- Mild
- Moderate
- Severe

**Business Rules**

Severity influences treatment urgency.

**Acceptance Criteria**

Severity displayed with explanation.

---

**FR-006 Weather Intelligence**

**Priority:** P1

**Description**

Retrieve weather forecast for user's farm.

**Data**

- Temperature
- Rain
- Humidity
- Wind

**Business Rules**

Recommendations consider:

- Rain probability
- Wind speed
- Temperature

**Acceptance Criteria**

Weather included in recommendations.

---

**FR-007 AI Decision Engine** ⭐

**Priority:** P0

**Description**

Core intelligence module.

Inputs:

- Disease
- Severity
- Weather
- Growth Stage
- Farm History
- Knowledge Base

Outputs:

- Recommendation
- Reason
- Confidence
- Next Action

**Example**

Disease:

Rust

Severity:

Moderate

Recommendation:

Spray tomorrow morning.

Reason:

Rain expected today.

Confidence:

92%

**Acceptance Criteria**

Recommendations are:

- Explainable
- Context-aware
- Generated under target latency

---

**FR-008 Explainability**

**Priority:** P0

Every recommendation shall include:

- Why
- Supporting evidence
- Confidence

Example

Spray tomorrow

Reason:

Rain expected tonight.

Humidity favors disease spread.

Morning spraying improves effectiveness.

---

**FR-009 RAG Knowledge Retrieval**

**Priority:** P0

Retrieve relevant agricultural documents before generating recommendations.

Knowledge Sources:

- Agriculture Department
- FAO
- PARC
- Crop manuals

Acceptance Criteria:

Recommendations reference retrieved knowledge rather than relying solely on model memory.

---

**FR-010 Treatment Verification**

**Priority:** P1

User scans pesticide bottle.

Outputs:

- Suitable
- Not suitable
- Alternative product
- Safety notes

Acceptance Criteria:

Product compatibility verified using knowledge base.

---

**FR-011 Voice Assistant**

**Priority:** P1

Recommendations can be spoken aloud.

Languages:

- Pashto
- Urdu
- English

Acceptance Criteria

User taps speaker icon.

Recommendation plays.

---

**FR-012 Scan History**

**Priority:** P1

Maintain historical records.

Each scan stores:

- Image
- Disease
- Recommendation

- Timestamp
- Farm

Acceptance Criteria

History searchable.

---

**FR-013 Farm Timeline**

Display chronological crop events.

Example

Day 1

Disease detected

↓

Day 4

Treatment applied

↓

Day 8

Recovery

↓

Day 20

Healthy

---

**FR-014 Community Disease Intelligence** ⭐

**Priority:** P2

With user consent:

Anonymous reports contribute to:

- Disease maps
- Hotspots
- Trends

Acceptance Criteria

Personally identifiable information is excluded.

---

**FR-015 Notifications**

Examples:

- Follow-up reminder
- Weather alert
- Disease outbreak nearby

---

**FR-016 Government Dashboard**

Displays

- Active outbreaks
- Disease frequency
- District statistics
- Heat maps

Restricted to authorized users.

---

**FR-017 Feedback**

Users can rate recommendations.

- Helpful
- Not Helpful

Feedback used to improve future versions.

---

**Functional Requirement Traceability**

| Feature | MVP |
|---|---|
| Authentication | ✅ |
| Farm Management | ✅ |
| Crop Scan | ✅ |
| Disease Detection | ✅ |
| Decision Engine | ✅ |
| Weather | ✅ |
| RAG | ✅ |
| Voice | ✅ |
| Product Verification | MVP+ |
| Community Intelligence | MVP |
| Dashboard | MVP+ |

---

**Chapter 8 — Non-Functional Requirements**

---

Unlike functional requirements,

these describe **how well** the system performs.

---

**8.1 Performance**

| Requirement | Target |
| --- | --- |
| Login | <2 sec |
| Image Upload | <5 sec |
| AI Recommendation | <10 sec |
| Dashboard Load | <3 sec |
| Search History | <2 sec |

---

**8.2 Availability**

System uptime:

99%

Scheduled maintenance outside peak hours.

---

**8.3 Reliability**

The platform should:

- Handle API failures gracefully.
- Retry temporary failures.
- Display meaningful error messages.

---

**8.4 Scalability**

Initial:

500 users

Future:

100,000+

Architecture must support:

Horizontal scaling.

Stateless backend.

---

**8.5 Security**

Requirements

HTTPS only

JWT Authentication

Password hashing

Role-based access

Input validation

Rate limiting

Audit logging

---

**8.6 Privacy**

User owns their data.

Community intelligence uses:

Anonymous data only.

Explicit consent required.

---

**8.7 Accessibility**

Support:

- Large buttons
- High contrast
- Keyboard navigation
- Screen readers (future)
- Voice playback
- Pashto
- Urdu

---

**8.8 Localization**

Languages

English

Urdu

Pashto

Future:

Regional dialect support.

---

**8.9 AI Safety**

Recommendations must:

Never claim certainty.

Show confidence.

Recommend experts when uncertain.

Avoid harmful instructions.

Ground responses using RAG.

---

**8.10 Explainability**

Every AI recommendation must answer:

- Why?
- Based on what?
- Confidence?
- Alternative?

---

**8.11 Compatibility**

Browsers:

Chrome

Firefox

Edge

Safari

Responsive:

Desktop

Tablet

Mobile

---

**8.12 Maintainability**

Backend:

Modular

Clean Architecture

REST APIs

Reusable services

Frontend:

Component-based React

---

**8.13 Observability**

System logs:

Authentication

Errors

API calls

Inference latency

Recommendation generation

Weather failures

---

**8.14 Disaster Recovery**

Daily backups.

Automatic database snapshots.

Restore procedures documented.

---

**8.15 Compliance**

System follows:

Responsible AI principles

Privacy-by-design

Least privilege access

Secure API practices

---

**8.16 Error Handling**

Examples

No Internet

↓

Show retry button

Unknown Disease

↓

Recommend agriculture officer

API Failure

↓

Fallback message

Low Confidence

↓

Explain uncertainty

---

**8.17 Monitoring Metrics**

Monitor:

Average response time

Daily active users

Scan success rate

Inference latency

API failures

Recommendation acceptance rate

Hotspot detection frequency

---

### 8.18 Quality Attributes

| Attribute | Target |
|---|---|
| Performance | High |
| Security | High |
| Reliability | High |
| Availability | High |
| Accessibility | High |
| Explainability | High |
| Scalability | Medium (MVP) |
| Offline Support | Future |

---

### Chapter Summary

By the end of Chapters 7 and 8, KissanDost AI has:

- Clearly defined system functionality.
- Measurable acceptance criteria.
- Performance expectations.
- Security and privacy requirements.
- AI safety principles.
- Operational quality targets.

This section serves as the implementation contract between product management, design, engineering, and QA.

---

### 🔍 Suggestions to strengthen the final PRD

As we move into the next chapters, I'd recommend adding:

1. **Requirement IDs linked to API endpoints** (traceability from requirement → implementation).
2. **MoSCoW prioritization** (Must, Should, Could, Won't) in addition to P0–P3.
3. **Sequence diagrams** for the scan and recommendation workflow.
4. **State diagrams** for scans, recommendations, and notifications.

These additions will make the final document feel much closer to an enterprise product specification.

## Chapter 9 — User Stories & Acceptance Criteria

---

### 9.1 Purpose

User stories describe the product from the user's perspective. They help designers, developers, and testers understand **what value** each feature delivers and how success will be measured.

---

### 9.2 User Roles

| Role | Description |
|---|---|
| Farmer | Primary user who scans crops and receives recommendations |
| Agriculture Officer | Reviews reports and disease trends |
| Government Analyst | Monitors regional disease intelligence |
| System Administrator | Manages users, knowledge base, and system health |

---

### Epic 1 — User Account & Farm Management

---

#### US-001 — Create Account

**As a** farmer
**I want** to register an account
**So that** my farm history and recommendations are saved.

**Acceptance Criteria**

- Email validation is performed.
- Duplicate accounts are prevented.
- User is logged in after successful registration.

---

**US-002 — Add a Farm**

**As a** farmer
**I want** to register my farm
**So that** recommendations are linked to the correct crop and location.

**Acceptance Criteria**

- Farm details are saved successfully.
- Multiple farms can be added.
- Farm can be edited later.

---

**Epic 2 — Crop Analysis**

---

**US-003 — Upload Crop Image**

**As a** farmer
**I want** to upload or capture a crop photo
**So that** the system can analyze it.

**Acceptance Criteria**

- Camera and gallery are supported.
- Unsupported file types are rejected.
- Preview is displayed before submission.

---

**US-004 — Detect Disease**

**As a** farmer
**I want** the system to identify crop diseases
**So that** I understand what is affecting my crop.

**Acceptance Criteria**

- Disease name displayed.

- Confidence score displayed.
- Unknown cases handled gracefully.

---

**US-005 — Estimate Severity**

**As a** farmer
**I want** to know how serious the disease is
**So that** I understand the urgency.

**Acceptance Criteria**

Severity displayed as:

- Healthy
- Mild
- Moderate
- Severe

---

**Epic 3 — AI Decision Support**

---

**US-006 — Receive Recommendation**

**As a** farmer
**I want** one clear recommendation
**So that** I know what action to take.

**Acceptance Criteria**

Recommendation includes:

- Action
- Reason
- Confidence
- Next step

---

**US-007 — Explain Recommendation**

**As a** farmer
**I want** to understand why the AI made its recommendation
**So that** I can trust it.

**Acceptance Criteria**

Explanation references:

- Disease
- Weather
- Severity
- Knowledge source

---

**US-008 — Weather-Based Advice**

**As a** farmer
**I want** recommendations that consider the weather
**So that** treatments are more effective.

**Acceptance Criteria**

Recommendation changes when:

- Rain is expected
- High wind is detected
- Extreme temperatures occur

---

**Epic 4 — Farm History**

---

**US-009 — View Previous Scans**

**As a** farmer
**I want** to review previous scans
**So that** I can track crop progress.

**Acceptance Criteria**

History displays:

- Date
- Crop

- Disease
- Recommendation

---

**US-010 — Track Recovery**

**As a** farmer
**I want** to compare previous and current scans
**So that** I know whether treatment worked.

---

**Epic 5 — Community Disease Intelligence**

---

**US-011 — Share Anonymous Reports**

**As a** farmer
**I want** to anonymously contribute disease reports
**So that** nearby farmers benefit from early warnings.

**Acceptance Criteria**

- User consent required.
- Personal information excluded.
- Location generalized to protect privacy.

---

**US-012 — Receive Nearby Alerts**

**As a** farmer
**I want** to receive alerts about disease outbreaks nearby
**So that** I can inspect my crops earlier.

---

**Epic 6 — Government Dashboard**

---

**US-013 — Monitor Disease Trends**

**As an** agriculture officer
**I want** to view disease trends by district
**So that** I can prioritize field visits.

---

**US-014 — Identify Hotspots**

**As a** government analyst
**I want** disease hotspots displayed on a map
**So that** resources can be allocated efficiently.

---

**Epic 7 — Feedback**

---

**US-015 — Rate Recommendation**

**As a** farmer
**I want** to rate AI recommendations
**So that** future recommendations improve.

---

**9.3 User Story Priority Matrix**

| Epic | Priority |
|---|---|
| Authentication | Must |
| Farm Management | Must |
| Crop Scan | Must |
| Disease Detection | Must |
| AI Recommendation | Must |
| Weather Intelligence | Must |
| History | Should |
| Voice Support | Should |
| Community Intelligence | Should |
| Government Dashboard | Could |
| Analytics | Could |

---

**9.4 Definition of Done**

A user story is complete when:

- Functional requirements are implemented.
- Acceptance criteria pass.
- Unit tests pass.
- Security checks pass.
- UI follows design guidelines.
- Product Owner approves.

---

**Chapter 10 — Complete UX Flows**

---

**10.1 UX Philosophy**

KissanDost AI follows a **"Photo First, Action First"** design philosophy.

Users should never have to navigate complex menus or understand technical agricultural terms.

**Design Principles**

- One primary action per screen.
- Minimal typing.
- Large touch targets.
- Local language support.
- Clear progress indicators.
- Explain every AI decision.

---

**10.2 Primary Navigation**

Home

│

├— Scan Crop

├— My Farms

├— History

├── Alerts

└── Profile

---

**10.3 First-Time User Flow**

Open App



Welcome



Choose Language



Create Account



Add Farm



Tutorial



Home Dashboard

---

**10.4 Crop Scan Flow (Core Journey)**

Home



Scan Crop



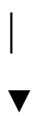
Camera / Gallery



Image Preview



AI Processing



Results



Recommendation



Save to History

---

**10.5 AI Processing Flow**

Upload Image



Image Validation



Disease Detection



Severity Estimation



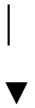
Weather Retrieval



Knowledge Retrieval (RAG)



Decision Engine



Recommendation Generated

---

**10.6 Recommendation Screen**

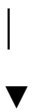
The recommendation page displays:

- Crop Name
- Disease
- Severity
- Confidence Score
- Recommended Action
- Best Time to Act
- Weather Summary
- Why This Recommendation?
- "Consult an Agriculture Officer" (if needed)
- Save / Share options

---

**10.7 Community Intelligence Flow**

Scan Completed



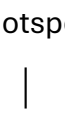
Ask for Consent



Anonymous Report Created



Hotspot Analysis

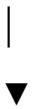


Community Dashboard Updated

---

**10.8 Government Dashboard Flow**

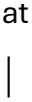
Login



District Overview



Heat Map



Disease Details



▼

Trend Analysis

|

▼

Download Reports

---

**10.9 Notification Flow**

Notifications may be triggered by:

- Follow-up reminder.
- Nearby disease outbreak.
- Weather conditions affecting treatment.
- New recommendation available.

---

**10.10 Error Flows**

**No Internet**

Upload Attempt

|

▼

Connection Failed

|

▼

Retry

---

**Unknown Disease**

AI Uncertain

|

▼

Explain Low Confidence

Recommend Agriculture Officer

---

**Image Quality Too Low**

Blur Detected



Ask User to Retake Image

---

**10.11 Accessibility Flow**

Users may:

- Switch language at any time.
- Listen to recommendations using voice playback.
- Increase text size (future).
- Use high-contrast mode (future).

---

**10.12 UX Success Metrics**

| Metric | Target |
|---|---|
| Time to first scan | <2 minutes |
| Scan completion rate | >90% |
| Recommendation understanding | >85% |
| Average task completion | <60 seconds |
| User satisfaction | >4.5/5 |

---

**10.13 Design Guidelines**

**Colors**

- Green → Healthy / Safe
- Yellow → Caution
- Red → Immediate Action
- Blue → Information

**Icons**

Use simple, universally recognizable icons for:

- Camera
- Farm
- Weather
- Alerts
- Voice
- History

**Typography**

- Large headings
- High contrast
- Readable on low-cost Android devices

---

**10.14 Future UX Enhancements**

- Offline scan queue.
- Voice-only interaction.
- AI chatbot for follow-up questions.
- Interactive treatment timeline.
- Personalized farming dashboard.

---

**Chapter Summary**

These chapters define **how users interact with KissanDost AI** and what outcomes they should expect. The UX prioritizes simplicity, trust, and speed, ensuring that even users with limited digital literacy can complete the primary task—capturing a crop photo and receiving a clear recommendation—with minimal effort.

**Chapter 11 — Wireframes**

**Goal:** Define the structure and layout of each major screen. These are low-fidelity wireframes intended to guide UI/UX design.

---

**11.1 Design Principles**

- **Photo-first interaction:** The primary action is always "Scan Crop."
- **Minimal cognitive load:** One main action per screen.
- **Pashto-first localization:** Easy language switching.
- **Explainable AI:** Every recommendation includes a "Why?" section.
- **Accessible UI:** Large buttons, readable fonts, high contrast.

---

**11.2 Home Dashboard**

---

☰ KissanDost AI 🔔 👤

Good Morning, Ahmad 🌱

Your Farm

Tomato Farm - Mardan

[📷 Scan Crop]

---

Recent Scan

✓ Tomato Leaf Blight

Recommendation

Spray tomorrow morning

[View Details]

---

Nearby Alerts

⚠ Leaf Blight detected

within 10 km

---

Bottom Navigation

Home | Scan | History | Profile

---

---

**11.3 Crop Scan Screen**

---

Take Crop Photo

+-------------------------+

|                         |

|    Camera Preview       |

|                         |

+--------------------------+

📷 Capture

OR

🖼 Upload Image

---

---

**11.4 AI Analysis Screen**

Analyzing...

✓ Detecting Crop

✓ Identifying Disease

✓ Checking Weather

✓ Retrieving Knowledge

✓ Generating Recommendation

Estimated Time

5-8 Seconds

---

**11.5 Recommendation Screen**

---

Crop

Tomato

Disease

Early Blight

Severity

Moderate

Confidence

92%

---

Recommended Action

Spray tomorrow morning.

---

Why?

• Rain expected today

• Moderate infection

• Best spray window

tomorrow 7-10 AM

---

Buttons

Save

Share

Voice

---

---

**11.6 Scan History**

History

Oct 12

Tomato

Early Blight

Recovered

---

Oct 2

Tomato

Healthy

---

Sept 18

Tomato

Leaf Spot

Recovered

---

**11.7 Government Dashboard**

---

KP Heat Map







---

Disease Frequency

Tomato Blight

45%

Rust

21%

Wilt

18%

---

District Statistics

Alerts

Reports

Export

---

---

### 11.8 Design System

**Primary Colors**

Green → Healthy

Orange → Warning

Red → High Risk

Blue → Information

---

**Icons**

📷 Scan

🌱 Crop

☁️ Weather

📊 Analytics

📍 Location

📢 Voice

---

**Typography**

Heading

Bold

18–24 px

Body

16 px

Buttons

16–18 px

---

### 11.9 Future Wireframes

- Marketplace
- AI Chat

- Satellite Monitoring
- Drone Reports
- Yield Dashboard

---

**Chapter 12 — AI Architecture**

---

**12.1 AI Philosophy**

KissanDost AI is **not a single AI model**.

It is an orchestration platform where multiple AI and rule-based components collaborate to produce one reliable recommendation.

This improves accuracy, explainability, and trust.

---

**12.2 High-Level AI Pipeline**

Image



Image Preprocessing



Vision Model



Disease Detection



Severity Estimation



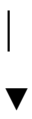
Weather Context



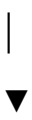
RAG Retrieval



Decision Engine



LLM Explanation



Final Recommendation

---

**12.3 AI Components**

**1. Computer Vision**

Purpose:

- Detect crop
- Detect disease
- Estimate severity

Candidate Models:

- Gemini Vision
- GPT-4.1 Vision (future)
- Open-source alternatives for offline deployment

---

**2. Large Language Model (LLM)**

Responsibilities:

- Generate explanations

- Simplify recommendations
- Translate responses
- Produce structured outputs

The LLM does **not** make decisions independently; it explains decisions produced by the system.

---

**3. Decision Engine**

Combines:

- Vision results
- Weather
- Growth stage
- RAG
- Farm history

Produces the recommended action.

---

**4. Weather Module**

Inputs:

- Rain probability
- Humidity
- Wind
- Temperature

Example:

Rain Tomorrow

↓

Delay Spray

---

**5. Explainability Module**

Every recommendation answers:

Why?

What evidence?

Confidence?

Next step?

---

**12.4 Confidence Strategy**

Overall confidence is calculated using:

- Vision confidence
- Knowledge relevance
- Weather completeness
- Decision consistency

Example:

Disease

96%

Knowledge

92%

Weather

100%

Final

95%

---

**12.5 AI Safety**

The AI must never:

❌ Recommend restricted chemicals without verification.

❌ Claim certainty when confidence is low.

❌ Ignore uncertainty.

Instead:

Confidence

52%

↓

Recommendation

Consult Agriculture Officer

---

**12.6 Explainable AI Example**

Instead of:

Spray Fungicide X.

The system returns:

Early Blight detected with high confidence. Moderate infection is visible on lower leaves. Rain is expected this evening, reducing spray effectiveness. Weather conditions tomorrow morning are more suitable. Based on agricultural guidance, spraying tomorrow between 7–10 AM is recommended.

---

**12.7 Future AI Roadmap**

- Multi-image analysis

- Video-based diagnosis
- Voice interaction
- Offline inference
- Drone image analysis
- Satellite imagery
- Personalized recommendations

---

**Chapter 13 — RAG (Retrieval-Augmented Generation) Architecture**

---

**13.1 Why RAG?**

General AI models can produce plausible but incorrect recommendations.

RAG ensures responses are grounded in trusted agricultural knowledge before the LLM generates an answer.

This reduces hallucinations and improves trust.

---

**13.2 Knowledge Sources**

- KP Agriculture Department advisories
- Pakistan Agricultural Research Council (PARC)
- FAO crop protection guides
- University extension publications
- Approved pesticide guidelines
- Weather data

Only vetted sources are indexed.

---

**13.3 RAG Pipeline**

User Question

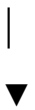
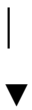


Image Analysis



Query Builder



Vector Search



Relevant Documents



Context Assembly



LLM



Grounded Response

---

**13.4 Knowledge Processing**

1. Collect trusted documents.
2. Convert to text.
3. Split into logical chunks.
4. Generate embeddings.
5. Store in a vector database.
6. Retrieve the most relevant chunks for each request.

---

### 13.5 Vector Database

Recommended options:

- PostgreSQL + pgvector (preferred for MVP)
- Pinecone (cloud alternative)
- Weaviate (future)
- Milvus (large-scale deployments)

For the hackathon, **pgvector** keeps the architecture simple by storing application data and embeddings in one database.

---

### 13.6 Prompt Structure

The LLM receives:

- Disease
- Severity
- Weather
- Growth stage
- Retrieved documents
- Farm history
- Output format

The prompt instructs the model to:

- Stay within retrieved context.
- Avoid unsupported advice.
- Explain recommendations clearly.
- Respond in the user's selected language.

---

### 13.7 RAG Safety Rules

The system must:

- Reject unsupported recommendations.
- Cite the source category internally.

- Avoid fabricating pesticide names.
- Escalate uncertain cases to agricultural officers.

---

**13.8 Benefits of RAG**

| Without RAG | With RAG |
|---|---|
| Relies on model memory | Uses trusted documents |
| Higher hallucination risk | Grounded recommendations |
| Difficult to update | Knowledge base updated independently |
| Generic responses | Localized, evidence-based guidance |

---

**13.9 Future Enhancements**

- Automatic synchronization with government advisories.
- Seasonal recommendation updates.
- Multilingual knowledge indexing.
- Version-controlled agricultural knowledge base.

**Chapter 14 — Decision Engine Design**

---

**14.1 Purpose**

The Decision Engine is the core intelligence layer of KissanDost AI. Instead of relying solely on an LLM, it combines structured inputs from multiple services to produce a safe, explainable, and context-aware recommendation.

This hybrid approach ensures recommendations are **consistent, auditable, and grounded in agricultural best practices**.

---

**14.2 Inputs**

| Source | Data |
|---|---|
| Vision AI | Crop, disease, confidence |

| Source | Data |
|---|---|
| Severity Module | Healthy, Mild, Moderate, Severe |
| Weather API | Rain, humidity, temperature, wind |
| Growth Stage | Seedling, Vegetative, Flowering, Fruiting, Harvest |
| RAG | Relevant agricultural guidance |
| Farm History | Previous scans and treatments |
| User Preferences | Language, notifications |

---

**14.3 Decision Flow**

Crop Image



Disease Detection



Severity Assessment



Weather Check



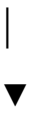
Growth Stage



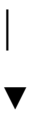
Retrieve Agricultural Guidance (RAG)



Decision Rules



Recommendation



Explanation

---

**14.4 Example Decision Rules**

| Condition | Recommendation |
|---|---|
| Mild disease + no rain | Monitor crop and inspect again in 3 days |
| Moderate disease + dry weather | Spray during recommended morning window |
| Severe disease | Immediate treatment and consult agriculture officer |
| Confidence < 70% | Request better image or expert review |
| Heavy rain forecast | Delay spraying until weather improves |

---

**14.5 Decision Priorities**

The engine evaluates information in this order:

1. AI confidence
2. Disease severity
3. Weather conditions
4. Crop growth stage
5. Official agricultural guidance
6. Farm history

This prioritization prevents a single AI prediction from overriding critical contextual information.

---

### 14.6 Recommendation Structure

Each recommendation includes:

- **Diagnosis**
- **Severity**
- **Recommended Action**
- **Best Time**
- **Reasoning**
- **Confidence**
- **Next Review Date**
- **Escalation Advice** (if applicable)

---

### 14.7 Explainability

Every recommendation is accompanied by a plain-language explanation.

**Example**

**Action:** Spray tomorrow between 7:00–10:00 AM.

**Why?**

- Moderate Early Blight detected.
- Rain expected today would reduce spray effectiveness.
- Weather tomorrow is favorable.
- Recommendation is based on trusted agricultural guidance.

---

### 14.8 Safety Rules

The Decision Engine must:

- Never recommend treatment when confidence is very low.
- Never recommend restricted products without verification.
- Clearly indicate uncertainty.
- Recommend consultation with an agriculture officer for severe or uncertain cases.

---

## Chapter 15 — Community Disease Intelligence

---

### 15.1 Purpose

Community Disease Intelligence transforms individual crop scans into anonymized regional insights.

Instead of helping only one farmer, KissanDost AI helps entire farming communities by identifying disease trends and potential outbreaks early.

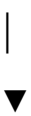
---

### 15.2 Workflow

Crop Scan



User Consent



Remove Personal Information



Store Anonymous Report



Trend Analysis



Hotspot Detection



Farmer & Government Alerts

---

### 15.3 Data Collected

With explicit user consent:

- District
- Approximate location (not exact GPS)
- Crop type
- Disease
- Severity
- Scan date

Not collected:

- Farmer name
- Phone number
- Exact location
- Personal notes

---

### 15.4 Hotspot Detection

The system continuously analyzes anonymous reports.

If disease frequency exceeds a configurable threshold within a region, the area is marked as a **potential hotspot**.

---

### 15.5 Example

| Village | Disease Reports | Status |
|---|---|---|
| Village A | 3 | Normal |
| Village B | 12 | Watch |
| Village C | 26 | High Risk |

---

### 15.6 Farmer Alerts

Example notification:

**Nearby Alert**

Tomato Early Blight cases have increased in your district during the past week. Inspect your crops regularly and monitor for symptoms.

---

### 15.7 Government Dashboard

Authorized users can view:

- Disease heat maps
- Trends by district
- Crop-wise statistics
- Weekly/monthly comparisons
- High-risk areas

---

### 15.8 Privacy Principles

Community intelligence is built on **Privacy by Design**:

- User consent is required.
- Reports are anonymized.
- Exact farm locations are never displayed publicly.
- Data is used only for agricultural monitoring.

---

### 15.9 Future Enhancements

- Seasonal outbreak prediction.
- AI-based risk forecasting.
- Integration with agriculture department advisory systems.
- Regional SMS alerts.

---

## Chapter 16 — System Architecture

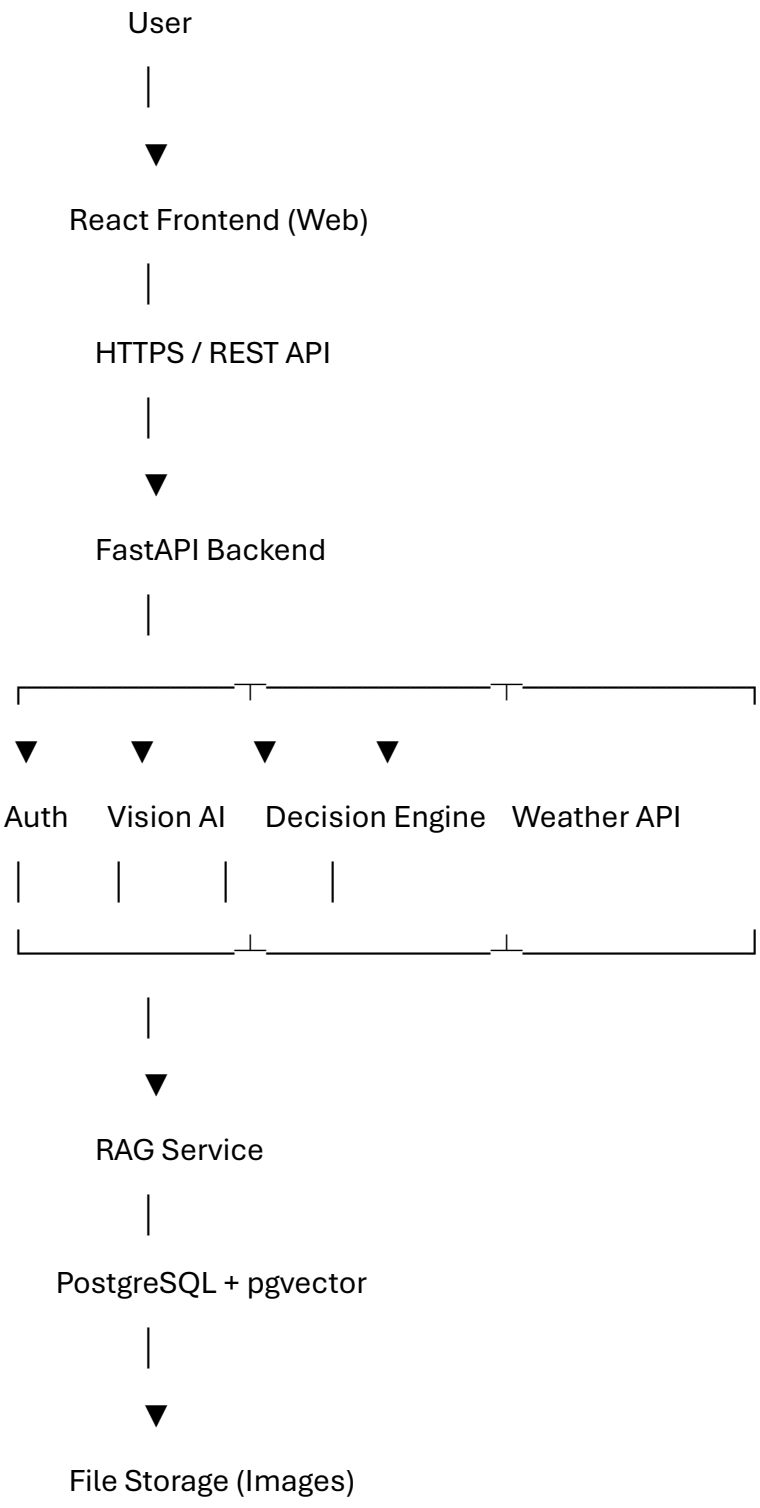
---

### 16.1 Architecture Overview

KissanDost AI follows a modular, service-oriented architecture that separates the user interface, business logic, AI services, and data layer. This design improves scalability, maintainability, and future extensibility.

---

**16.2 High-Level Architecture**




---

**16.3 Frontend**

Technology:

- React
- TypeScript
- Tailwind CSS

Responsibilities:

- Authentication
- Image upload
- Recommendation display
- History
- Dashboard
- Localization

---

**16.4 Backend**

Technology:

- FastAPI
- Python

Responsibilities:

- Authentication
- Business logic
- AI orchestration
- API management
- Recommendation generation

---

**16.5 AI Layer**

Components:

- Vision Model
- Decision Engine

- RAG Service
- Translation Module
- Explainability Module

The backend coordinates these services to produce a single recommendation.

---

**16.6 Database Layer**

Primary Database:

- PostgreSQL

Extensions:

- pgvector (embeddings)

Stores:

- Users
- Farms
- Scans
- Recommendations
- Anonymous reports
- Audit logs

---

**16.7 Storage**

Object storage is used for:

- Uploaded crop images
- Processed images (optional)
- Generated reports

Images are linked to scan records rather than stored directly in the database.

---

**16.8 External Services**

| Service | Purpose |
|---|---|
| Gemini Vision | Disease detection |
| Gemini LLM | Explanation generation |
| Open-Meteo | Weather forecast |
| Supabase Auth (optional) | Authentication |
| PostgreSQL | Application data |
| pgvector | Knowledge retrieval |

---

**16.9 API Communication**

Frontend



FastAPI



- Authentication
- Scan Service
- AI Service
- RAG Service
- Weather Service
- Notification Service

---

**16.10 Security Architecture**

- HTTPS for all communication.
- JWT-based authentication.
- Role-Based Access Control (RBAC).
- Encrypted credentials.
- Input validation.

- API rate limiting.
- Audit logging for sensitive operations.

---

**16.11 Logging & Monitoring**

System logs:

- User authentication
- Image uploads
- AI processing time
- API failures
- Recommendation generation
- System errors

Monitoring metrics:

- Average response time
- AI latency
- Error rate
- Daily scans
- Active users

---

**16.12 Scalability**

The architecture supports future growth through:

- Stateless backend services.
- Horizontal API scaling.
- Independent AI services.
- Database indexing and caching.
- Modular microservice migration if needed.

---

**16.13 Future Architecture**

Potential additions:

- Mobile applications.
- Offline AI inference.
- Drone image processing.
- IoT sensor integration.
- Satellite imagery analysis.
- Edge deployment for remote areas.

**Chapter 17 — Database Design**

---

### 17.1 Database Overview

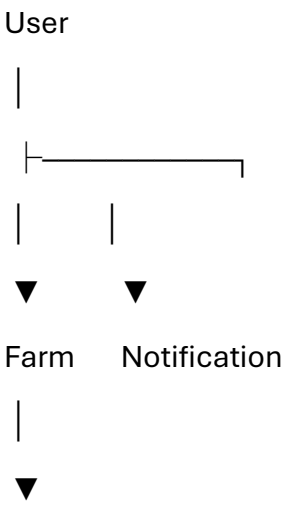
KissanDost AI uses **PostgreSQL** as the primary relational database. It stores operational data such as users, farms, crop scans, recommendations, and anonymous community reports. The **pgvector** extension is used to store document embeddings for the RAG knowledge base.

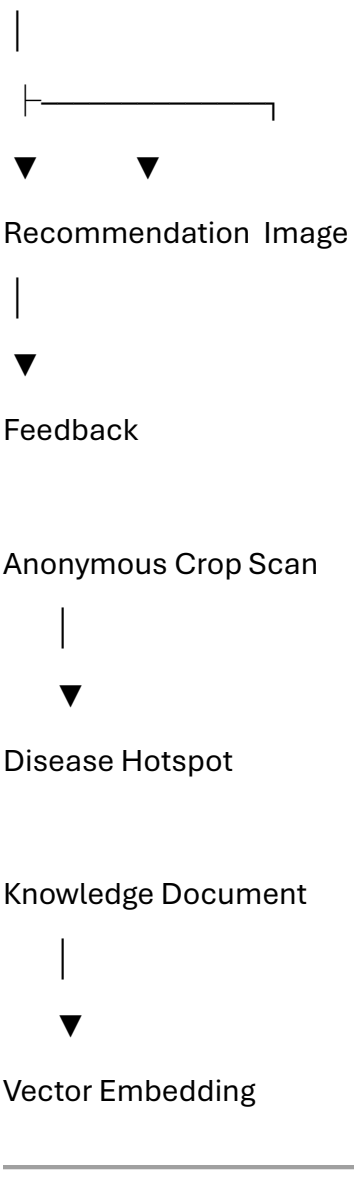
**Database Goals**

- Maintain data integrity
- Support efficient queries
- Scale for future growth
- Separate operational and AI knowledge data

---

### 17.2 Entity Relationship Diagram (Conceptual)






---

**17.3 Core Tables**

**Users**

| Field | Type |
|---|---|
| id | UUID |
| full_name | VARCHAR |
| email | VARCHAR |
| password_hash | TEXT |
| role | ENUM |
| language | ENUM |

| Field | Type |
| --- | --- |
| created_at | TIMESTAMP |

---

**Farms**

| Field | Type |
| --- | --- |
| id | UUID |
| user_id | UUID |
| farm_name | VARCHAR |
| district | VARCHAR |
| village | VARCHAR |
| crop_type | VARCHAR |
| area | DECIMAL |
| planting_date | DATE |

Relationship:

**One User → Many Farms**

---

**Crop Scans**

| Field | Type |
| --- | --- |
| id | UUID |
| farm_id | UUID |
| image_url | TEXT |
| crop_name | VARCHAR |
| disease_name | VARCHAR |
| confidence | DECIMAL |
| severity | ENUM |

| Field | Type |
| --- | --- |
| created_at | TIMESTAMP |

---

**Recommendations**

| Field | Type |
| --- | --- |
| id | UUID |
| scan_id | UUID |
| recommendation | TEXT |
| explanation | TEXT |
| confidence | DECIMAL |
| weather_snapshot | JSON |
| generated_at | TIMESTAMP |

---

**Notifications**

| Field | Type |
| --- | --- |
| id | UUID |
| user_id | UUID |
| title | VARCHAR |
| message | TEXT |
| type | ENUM |
| is_read | BOOLEAN |
| created_at | TIMESTAMP |

---

**Feedback**

| Field | Type |
| --- | --- |
| id | UUID |
| recommendation_id | UUID |
| rating | INTEGER |
| comments | TEXT |
| created_at | TIMESTAMP |

---

**Community Reports**

Stores **anonymous** data only.

| Field | Type |
| --- | --- |
| id | UUID |
| district | VARCHAR |
| crop | VARCHAR |
| disease | VARCHAR |
| severity | ENUM |
| reported_at | TIMESTAMP |

No personally identifiable information is stored in this table.

---

**Knowledge Base**

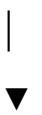
| Field | Type |
| --- | --- |
| id | UUID |
| source | VARCHAR |
| title | VARCHAR |
| chunk_text | TEXT |
| embedding | VECTOR |

| Field | Type |
| --- | --- |
| updated_at | TIMESTAMP |

---

### 17.4 Relationships

User (1)



Farm (N)



Crop Scan (N)



Recommendation (1)



Feedback (N)

---

### 17.5 Indexing Strategy

Indexes should be created for:

- email
- user_id
- farm_id
- district
- disease_name
- created_at
- vector embeddings

---

### 17.6 Data Retention

| Data | Retention |
|---|---|
| User profile | Until account deletion |
| Crop scans | User controlled |
| Recommendations | User controlled |
| Community reports | Long-term (anonymous) |
| Logs | 90 days |

---

### 17.7 Backup Strategy

- Daily automated backups
- Weekly full snapshots
- Point-in-time recovery (where supported)
- Off-site encrypted backup storage

---

## Chapter 18 — API Specification

---

### 18.1 API Overview

The backend exposes RESTful APIs secured using JWT authentication.

**Base URL**

/api/v1

---

### 18.2 Authentication APIs

**Register**

POST /auth/register

Request


```
{
  "name": "Muhammad Ahmad",
```

"email": "user@example.com",

"password": "********"

}

Response

{

"success": true,

"userId": "...",

"token": "JWT_TOKEN"

}

---

**Login**

POST /auth/login

Returns:

- JWT Token
- User Profile

---

**18.3 Farm APIs**

**Create Farm**

POST /farms

**List Farms**

GET /farms

**Update Farm**

PUT /farms/{id}

**Delete Farm**

DELETE /farms/{id}

---

**18.4 Crop Scan APIs**

**Upload Image**

POST /scans/upload

Multipart form-data:

- Image
- Farm ID

Response

{

"scanId":"..."

}

---

**Analyze Scan**

POST /scans/analyze/{scanId}

Returns:

- Disease
- Severity
- Confidence
- Recommendation

---

**Get Scan History**

GET /scans/history

---

**18.5 Recommendation API**

GET /recommendations/{scanId}

Response

{

"crop":"Tomato",

"disease":"Early Blight",

"severity":"Moderate",

"recommendation":"Spray tomorrow morning.",

"confidence":92

}

---

**18.6 Feedback API**

POST /feedback

Allows users to rate recommendations.

---

**18.7 Notification APIs**

GET /notifications

PATCH /notifications/{id}

Marks notification as read.

---

**18.8 Government Dashboard APIs**

GET /analytics/districts

GET /analytics/hotspots

GET /analytics/trends

Access restricted to authorized roles.

---

**18.9 Error Response Format**

{

"success":false,

"error":"INVALID_IMAGE",

"message":"Unsupported image format."

}

---

**18.10 API Status Codes**

| Code | Meaning |
|---|---|
| 200 | Success |
| 201 | Created |
| 400 | Bad Request |
| 401 | Unauthorized |
| 403 | Forbidden |
| 404 | Not Found |
| 429 | Too Many Requests |
| 500 | Internal Server Error |

---

### 18.11 API Versioning

The API uses URI versioning.

Example:

/api/v1

Future breaking changes will be released under /api/v2.

---

## Chapter 19 — Security & Privacy

---

### 19.1 Security Objectives

The platform must:

- Protect user accounts.
- Protect farm data.
- Ensure AI recommendations are trustworthy.
- Preserve user privacy.
- Comply with secure development practices.

---

### 19.2 Authentication

Supported methods:

- Email & Password
- Google Sign-In (future)

Passwords are hashed using a strong one-way algorithm (e.g., Argon2 or bcrypt).

---

**19.3 Authorization**

Role-Based Access Control (RBAC):

| Role | Permissions |
|---|---|
| Farmer | Manage own farms and scans |
| Agriculture Officer | View assigned regional insights |
| Government Analyst | View aggregated dashboards |
| Administrator | System administration |

Users may only access resources they are authorized to view.

---

**19.4 Encryption**

- HTTPS/TLS for all communications.
- Passwords stored as hashes.
- Sensitive configuration stored in environment variables.
- Database backups encrypted at rest.

---

**19.5 Privacy**

KissanDost AI follows **Privacy by Design**.

Principles:

- Collect only necessary data.
- Obtain user consent before contributing anonymous reports.
- Allow users to delete their data.
- Never share personally identifiable information in community analytics.

---

### 19.6 AI Safety

To reduce harmful outputs:

- Recommendations are grounded using RAG.
- Low-confidence predictions trigger caution messages.
- The AI avoids unsupported chemical recommendations.
- Users are advised to consult agricultural officers for uncertain or severe cases.

---

### 19.7 Threat Model

| Threat | Mitigation |
|---|---|
| Account takeover | Strong password hashing, JWT expiry |
| SQL Injection | Parameterized queries / ORM |
| Cross-Site Scripting (XSS) | Input validation and output encoding |
| Cross-Site Request Forgery (CSRF) | CSRF protection where applicable |
| API abuse | Rate limiting |
| Prompt injection | Strict prompt templates and context isolation |
| Hallucinated advice | RAG grounding + confidence thresholds |

---

### 19.8 Audit Logging

The following events are logged:

- User login
- Failed authentication attempts
- Recommendation generation
- Administrative actions
- Security-related events

Logs are protected against unauthorized modification.

---

### 19.9 Incident Response

In the event of a security incident:

1. Detect and isolate affected systems.
2. Preserve logs for investigation.
3. Assess user impact.
4. Apply fixes and verify integrity.
5. Notify affected stakeholders if required.

---

### 19.10 Compliance Principles

The platform is designed around:

- Least privilege access.
- Secure defaults.
- Privacy by design.
- Responsible AI.
- Transparent decision-making.

---

### 19.11 Security Checklist

| Control | Status |
| --- | --- |
| HTTPS Everywhere | ✅ |
| Password Hashing | ✅ |
| JWT Authentication | ✅ |
| RBAC | ✅ |
| Input Validation | ✅ |
| Rate Limiting | ✅ |
| Audit Logs | ✅ |
| Encrypted Backups | ✅ |

| Control | Status |
| --- | --- |
| AI Confidence Display | ✅ |
| User Consent | ✅ |

## Chapter 20 — Tech Stack Justification

---

### 20.1 Technology Selection Principles

The technology stack was selected based on the following criteria:

- Rapid hackathon development
- AI ecosystem compatibility
- Scalability
- Maintainability
- Open-source preference where practical
- Strong developer community
- Cloud deployment readiness

---

### 20.2 Frontend

**Technology**

- **React**
- **TypeScript**
- **Tailwind CSS**
- **Vite**

**Why React?**

- Component-based architecture
- Large ecosystem
- Easy integration with REST APIs
- Excellent developer productivity

**Why TypeScript?**

- Type safety
- Better maintainability
- Reduced runtime errors
- Improved IDE support

**Why Tailwind CSS?**

- Rapid UI development
- Consistent design system
- Lightweight production builds

---

**20.3 Backend**

**Technology**

- **FastAPI**
- **Python**

**Why FastAPI?**

- High performance
- Automatic OpenAPI documentation
- Native async support
- Excellent AI ecosystem compatibility
- Easy request validation

---

**20.4 Artificial Intelligence**

**Vision Model**

**Google Gemini Vision**

Reasons:

- Strong multimodal understanding
- Reliable image interpretation
- Rapid integration through APIs

---

**Language Model**

**Gemini 2.5**

Responsibilities:

- Explain recommendations
- Summarize retrieved knowledge
- Translate responses
- Generate structured outputs

---

**Why Not Fine-Tune a Model?**

Hackathon constraints make API-based foundation models more practical. The architecture remains modular so models can be replaced in future deployments.

---

**20.5 Database**

**PostgreSQL**

Reasons:

- Mature relational database
- ACID compliance
- Excellent indexing
- Reliable backup support

---

**pgvector**

Reasons:

- Native vector similarity search
- Keeps structured and vector data together
- Simplifies MVP architecture

---

**20.6 Object Storage**

Purpose:

- Store uploaded crop images
- Store generated reports
- Avoid bloating the relational database

Potential providers:

- Supabase Storage
- AWS S3
- Cloudflare R2

---

**20.7 Weather Data**

Preferred provider:

- **Open-Meteo API**

Reasons:

- Free access
- No API key required for many use cases
- Forecast data sufficient for MVP

---

**20.8 Authentication**

Preferred:

- JWT

Future:

- Google Sign-In
- OTP login

---

**20.9 Deployment**

Backend:

- Docker
- Railway / Render / Google Cloud Run

Frontend:

- Vercel

Database:

- Supabase PostgreSQL

---

**20.10 Alternative Technologies Considered**

| Layer | Selected | Alternative | Reason |
|---|---|---|---|
| Frontend | React | Next.js | Simpler MVP |
| Backend | FastAPI | Django | Lower overhead |
| Database | PostgreSQL | MongoDB | Better relational modeling |
| Vector DB | pgvector | Pinecone | Lower operational complexity |
| AI | Gemini | OpenAI / Claude | Flexible model abstraction |

---

**20.11 Final Tech Stack**

| Layer | Technology |
|---|---|
| Frontend | React + TypeScript + Tailwind |
| Backend | FastAPI |
| AI | Gemini Vision + Gemini 2.5 |
| Database | PostgreSQL |
| Vector Search | pgvector |
| Storage | Supabase Storage |
| Weather | Open-Meteo |
| Authentication | JWT |
| Deployment | Docker + Vercel + Railway |

---

**Chapter 21 — Deployment Architecture**

---

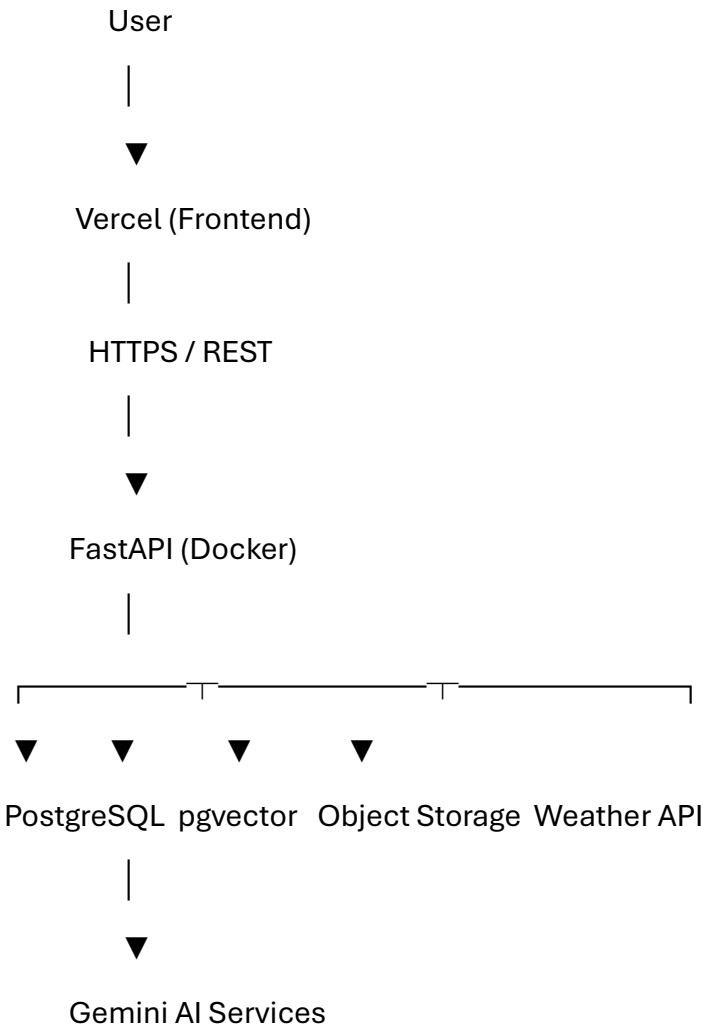
### 21.1 Deployment Goals

The deployment architecture should provide:

- High availability
- Secure communication
- Horizontal scalability
- Easy updates
- Low operational cost

---

### 21.2 Production Architecture




---

### 21.3 Containerization

Every backend service runs inside Docker.

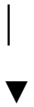
Benefits:

- Consistent environments
- Easy deployment
- Portable infrastructure
- Simplified scaling

---

**21.4 CI/CD Pipeline**

GitHub Push



GitHub Actions



Run Tests



Build Docker Image



Deploy Backend



Deploy Frontend

---

**21.5 Environment Configuration**

Separate environments:

- Development

- Staging
- Production

Environment variables include:

- API keys
- Database URL
- JWT secret
- Storage credentials

Secrets are never committed to source control.

---

**21.6 Monitoring**

Track:

- API latency
- AI response time
- Database health
- Error rate
- Storage usage
- Active users

Recommended tools:

- Grafana
- Prometheus
- Sentry

---

**21.7 Scaling Strategy**

**Horizontal Scaling**

- Multiple FastAPI instances behind a load balancer
- Stateless API design
- Shared PostgreSQL database

**Vertical Scaling**

Increase CPU/RAM for AI-intensive workloads when needed.

---

**21.8 Disaster Recovery**

- Automated backups
- Multi-region storage (future)
- Restore testing
- Infrastructure-as-Code for reproducible deployments

---

**21.9 Future Deployment**

Potential additions:

- Mobile apps
- Offline sync
- Edge AI
- Kubernetes for large-scale deployments

---

**Chapter 22 — KP Pilot Plan**

---

**22.1 Pilot Objective**

Validate that KissanDost AI improves crop decision-making for farmers in Khyber Pakhtunkhwa while collecting feedback for future enhancements.

---

**22.2 Pilot Scope**

**Duration:** 90 Days

**Initial Region:**

- Mardan *(recommended for MVP due to accessibility and agricultural activity)*

Future expansion:

- Swabi
- Charsadda

- Nowshera
- ARDC merged districts

---

**22.3 Stakeholders**

- Farmers
- Agriculture Extension Officers
- KP Agriculture Department
- Local universities
- Research institutions

---

**22.4 Pilot Phases**

**Phase 1 – Preparation (Weeks 1–2)**

- Configure infrastructure
- Populate RAG knowledge base
- Recruit pilot participants
- Conduct training sessions

---

**Phase 2 – Active Pilot (Weeks 3–10)**

Participants:

- Farmers use the application for real crop monitoring.
- Agriculture officers review reported trends.
- Collect usability and performance feedback.

---

**Phase 3 – Evaluation (Weeks 11–12)**

Measure:

- AI accuracy
- User satisfaction
- System reliability

- Community reporting participation

Prepare a final pilot report with recommendations for scaling.

---

**22.5 Success Metrics**

| Metric | Target |
|---|---|
| Farmers onboarded | 100+ |
| Crop scans completed | 1,000+ |
| Recommendation response time | <10 sec |
| User satisfaction | >85% |
| Community participation | >60% of users (opt-in) |
| System uptime | >99% |

---

**22.6 Data Collection**

Collect:

- Number of scans
- Recommendation acceptance rate
- Feedback ratings
- Outbreak reports
- AI confidence distribution

No personally identifiable information is included in public analytics.

---

**22.7 Risks During Pilot**

| Risk | Mitigation |
|---|---|
| Poor internet connectivity | Compress uploads and retry logic |
| Low adoption | On-site training and awareness |
| Low-quality images | In-app capture guidance |

| Risk | Mitigation |
| --- | --- |
| False positives | Human review for low-confidence cases |
| Weather API downtime | Cached forecast and graceful fallback |

---

### 22.8 Expansion Strategy

Following a successful pilot:

1. Expand across additional KP districts.
2. Partner with the KP Agriculture Department.
3. Increase crop coverage.
4. Introduce multilingual voice support.
5. Explore national deployment.

---

### 22.9 Expected Outcomes

By the end of the pilot, KissanDost AI aims to demonstrate:

- Faster crop disease identification.
- More informed treatment decisions.
- Earlier detection of regional outbreaks.
- Increased trust in AI-assisted agriculture.
- A validated foundation for province-wide deployment.

## Chapter 23 — Testing Strategy

---

### 23.1 Testing Objectives

The testing strategy ensures that KissanDost AI is:

- Functionally correct
- Secure
- Reliable

- Performant
- User-friendly
- AI-safe

Testing will be integrated throughout the development lifecycle rather than performed only at the end.

---

### 23.2 Testing Pyramid

Manual Acceptance

▲

Integration Tests

▲

Unit Tests

- **Unit Tests:** Validate individual functions and services.
- **Integration Tests:** Verify communication between modules.
- **Acceptance Tests:** Ensure the application meets business requirements.

---

### 23.3 Unit Testing

**Scope**

- Authentication
- Farm Management
- Scan Processing
- Decision Engine
- Weather Service
- Notification Service

**Target**

- Minimum **80% code coverage** for business logic.

---

### 23.4 Integration Testing

Validate interactions between:

- Frontend ↔ Backend
- Backend ↔ Gemini APIs
- Backend ↔ Weather API
- Backend ↔ PostgreSQL
- Backend ↔ RAG Service

---

**23.5 End-to-End (E2E) Testing**

Representative user journeys:

1. Register → Add Farm → Upload Scan → Receive Recommendation.
2. View Scan History.
3. Receive Nearby Disease Alert.
4. Submit Feedback.

Success is measured by completing each flow without critical failures.

---

**23.6 AI Validation**

Since AI outputs are probabilistic, testing focuses on:

- Correct structured output format.
- Presence of explanations.
- Confidence score display.
- Proper handling of uncertain predictions.
- Consistency across repeated evaluations.

---

**23.7 RAG Validation**

Verify that:

- Retrieved documents are relevant.
- Responses remain grounded in retrieved content.
- Missing knowledge results in uncertainty rather than fabricated answers.

- Knowledge updates are reflected in future recommendations.

---

**23.8 Performance Testing**

| Metric | Target |
|---|---|
| Login | <2 sec |
| Image Upload | <5 sec |
| Recommendation | <10 sec |
| Dashboard Load | <3 sec |

Load testing should simulate increasing concurrent users to identify bottlenecks.

---

**23.9 Security Testing**

The following will be verified:

- Authentication
- Authorization
- SQL Injection protection
- XSS prevention
- API rate limiting
- JWT validation
- Secure file uploads

---

**23.10 Usability Testing**

Pilot participants will evaluate:

- Ease of registration.
- Simplicity of crop scanning.
- Clarity of recommendations.
- Overall satisfaction.

Target satisfaction score:

**4.5 / 5 or higher**

---

### 23.11 Bug Classification

| Severity | Example |
|---|---|
| Critical | System unavailable |
| High | AI recommendation cannot be generated |
| Medium | Notification failure |
| Low | UI alignment issue |

---

### 23.12 Release Criteria

The MVP is considered ready when:

- No critical bugs remain.
- All core user stories pass acceptance tests.
- Performance targets are met.
- Security review is completed.
- Pilot stakeholders approve deployment.

---

## Chapter 24 — Product Roadmap

---

### 24.1 Product Vision

Develop KissanDost AI into Pakistan's most trusted AI-powered agricultural decision support platform, expanding from disease diagnosis to comprehensive farm intelligence.

---

### 24.2 Phase 1 – Hackathon MVP (0–3 Months)

**Goals**

- User authentication
- Farm registration

- Crop image upload
- Disease detection
- Weather integration
- AI recommendations
- RAG knowledge base
- Community Disease Intelligence
- Basic government dashboard

**Deliverable:** Working prototype demonstrating the end-to-end workflow.

---

### 24.3 Phase 2 – Pilot Deployment (3–6 Months)

**Goals**

- Deploy with selected farmers in KP.
- Improve AI prompts using pilot feedback.
- Expand crop and disease coverage.
- Enhance multilingual support.
- Strengthen analytics.

---

### 24.4 Phase 3 – Regional Expansion (6–12 Months)

**Goals**

- Expand across Khyber Pakhtunkhwa.
- Partner with the Agriculture Department.
- Launch Android application.
- Add offline data synchronization.
- Improve hotspot detection accuracy.

---

### 24.5 Phase 4 – National Platform (1–2 Years)

**Goals**

- Support all provinces of Pakistan.

- Add additional crop categories.
- Introduce satellite and drone integration.
- Expand government reporting capabilities.

---

### 24.6 Phase 5 – Intelligent Agriculture Platform (2+ Years)

**Vision**

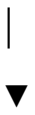
Transform KissanDost AI into a complete agricultural intelligence ecosystem including:

- Yield forecasting
- Pest prediction
- Smart irrigation recommendations
- Fertilizer optimization
- Carbon footprint reporting
- Climate adaptation insights

---

### 24.7 Roadmap Timeline

Hackathon MVP



Pilot



KP Expansion



Pakistan



AI Agriculture Platform

### 24.8 Success Milestones

| Stage | Milestone |
|---|---|
| MVP | First successful AI recommendation |
| Pilot | 100+ farmers onboarded |
| Regional | 10,000 crop scans |
| National | Multi-province deployment |
| Platform | Full farm intelligence ecosystem |

---

## Chapter 25 — Risk Analysis

---

### 25.1 Purpose

This chapter identifies key risks associated with product development, deployment, AI behavior, and user adoption, along with mitigation strategies.

---

### 25.2 Technical Risks

| Risk | Impact | Mitigation |
|---|---|---|
| AI API downtime | High | Retry logic, graceful fallback |
| Weather API unavailable | Medium | Cached forecasts |
| Slow inference | High | Optimize prompts, asynchronous processing |
| Database overload | Medium | Indexing, connection pooling |

---

### 25.3 AI Risks

| Risk | Mitigation |
|---|---|
| Hallucinated recommendations | RAG grounding + confidence thresholds |
| Incorrect disease detection | Human review for uncertain cases |

| Risk | Mitigation |
| --- | --- |
| Model changes by provider | Abstract AI layer for model replacement |
| Prompt injection | Strict prompt templates and validation |

---

**25.4 Data Risks**

| Risk | Mitigation |
| --- | --- |
| Data loss | Automated backups |
| Unauthorized access | RBAC + encryption |
| Privacy violations | Anonymization and user consent |
| Inaccurate knowledge | Curated and versioned knowledge base |

---

**25.5 User Risks**

| Risk | Mitigation |
| --- | --- |
| Low digital literacy | Simple UI, multilingual support, voice guidance |
| Poor image quality | In-app capture guidance and validation |
| Misinterpretation of advice | Explainable recommendations with confidence scores |

---

**25.6 Operational Risks**

| Risk | Mitigation |
| --- | --- |
| Low adoption | Farmer awareness campaigns and training |
| Limited internet access | Image compression and retry mechanisms |
| Funding constraints | Open-source stack and phased deployment |

---

**25.7 Regulatory & Ethical Risks**

- AI should support—not replace—agricultural experts.
- Recommendations must remain transparent and explainable.

- User consent is mandatory for community data sharing.
- Government analytics must use aggregated, anonymized information only.

---

**25.8 Risk Matrix**

| Probability | Impact | Example |
|---|---|---|
| High | High | AI service outage |
| High | Medium | Low-quality image uploads |
| Medium | High | Incorrect disease classification |
| Low | High | Data breach |
| Low | Medium | Weather API downtime |

---

**25.9 Business Continuity**

If critical services fail:

1. Notify the user.
2. Preserve uploaded data.
3. Retry automatically where appropriate.
4. Offer manual guidance when AI cannot provide a confident recommendation.

---

**25.10 Overall Risk Assessment**

The project presents **moderate technical risk** due to dependence on external AI and weather services, but **low implementation risk** because it uses proven technologies, modular architecture, and phased deployment. With proper testing, monitoring, and governance, KissanDost AI is well positioned for a successful MVP and pilot rollout.

**Chapter 26 — Demo Script**

---

**26.1 Objective**

Demonstrate that **KissanDost AI** solves a real agricultural problem using explainable AI, weather intelligence, and community-driven disease monitoring.

**Target Demo Duration: 5–7 minutes**

---

**26.2 Demo Flow**

**Step 1 – Problem (30 seconds)**

**Presenter**

"Smallholder farmers often identify crop diseases too late or receive inconsistent advice from multiple sources. Existing tools usually stop at disease detection, leaving farmers uncertain about what to do next."

---

**Step 2 – Introduce KissanDost AI (30 seconds)**

"KissanDost AI is an AI-powered agricultural decision support platform. Instead of simply identifying diseases, it analyzes the crop image, checks weather conditions, retrieves trusted agricultural guidance, and provides one clear, explainable recommendation."

---

**Step 3 – User Journey (2 minutes)**

1. Login.
2. Open farm.
3. Capture crop image.
4. Upload image.
5. AI analysis begins.
6. Recommendation appears.

Highlight:

- Disease
- Severity
- Weather
- Confidence
- Explanation

---

**Step 4 – Explainability (45 seconds)**

Open the **"Why this recommendation?"** section.

Explain that the recommendation is based on:

- Image analysis
- Weather
- Trusted agricultural knowledge
- Decision Engine

rather than only an LLM response.

---

**Step 5 – Community Disease Intelligence (45 seconds)**

Display the dashboard.

Show:

- Anonymous reports.
- District hotspot.
- Nearby disease alert.

Explain how this benefits both farmers and agricultural authorities.

---

**Step 6 – Technical Architecture (45 seconds)**

Show the architecture diagram.

Briefly explain:

- Vision AI
- RAG
- Decision Engine
- FastAPI
- PostgreSQL + pgvector

---

**Step 7 – Closing (30 seconds)**

"KissanDost AI empowers farmers with trusted, explainable recommendations while helping entire communities detect outbreaks earlier through anonymous disease intelligence."

---

**26.3 Live Demo Checklist**

- Stable internet connection.
- Test AI API before presentation.
- Sample crop images ready.
- Backup screenshots available.
- Backup video available.
- API keys configured.
- Demo account prepared.

---

**26.4 Demo Success Criteria**

The audience should understand:

- The problem.
- The AI workflow.
- Why the recommendation is trustworthy.
- The community impact.
- The technical feasibility.

---

**Chapter 27 — Judge Q&A**

---

**Q1. Why use AI?**

**Answer**

Disease diagnosis and treatment recommendations depend on multiple variables such as image features, weather, severity, and agricultural guidance. AI enables scalable analysis and personalized recommendations that would be difficult to automate using traditional rule-based software alone.

---

**Q2. Why not just use ChatGPT?**

**Answer**

ChatGPT is a general-purpose assistant.

KissanDost AI combines:

- Vision AI
- Weather intelligence
- RAG
- Decision Engine
- Farm history
- Community intelligence

The LLM explains recommendations rather than making unsupported decisions.

---

**Q3. How do you reduce hallucinations?**

**Answer**

We use Retrieval-Augmented Generation (RAG) with trusted agricultural documents, confidence thresholds, and structured prompts. Low-confidence predictions trigger caution messages instead of overconfident advice.

---

**Q4. How is user privacy protected?**

**Answer**

Community reporting is opt-in. Personally identifiable information is never included in community analytics. Only anonymized, aggregated data is used for hotspot detection.

---

**Q5. What makes this different from Plantix?**

**Answer**

Plantix focuses primarily on disease identification.

KissanDost AI extends the workflow by adding:

- Weather-aware recommendations.

- Explainable AI.
- Farm history.
- Community Disease Intelligence.
- Government analytics.
- RAG-grounded recommendations.

---

**Q6. Can this work offline?**

**Answer**

The MVP requires internet access because it relies on cloud AI services. Future versions will explore offline synchronization and lightweight on-device inference where practical.

---

**Q7. How will you measure success?**

**Answer**

Key metrics include:

- Recommendation response time.
- User satisfaction.
- Scan completion rate.
- Community participation.
- Pilot adoption.
- Recommendation acceptance rate.

---

**Q8. Why is this relevant to KP?**

**Answer**

Agriculture is a major contributor to the regional economy. Early disease detection and timely recommendations can improve productivity while supporting agriculture officers with regional insights.

---

**Q9. Is this replacing agriculture experts?**

**Answer**

No.

KissanDost AI is a decision support system designed to extend expert reach. For uncertain or severe cases, it recommends consulting agricultural professionals.

---

**Q10. What is your long-term vision?**

**Answer**

To build Pakistan's most trusted AI-powered agricultural decision support platform, evolving into a comprehensive farm intelligence ecosystem.

---

**Chapter 28 — Future Scope**

---

**Short-Term**

- Android application.
- More supported crops.
- Expanded disease database.
- Improved multilingual support.
- Better analytics.

---

**Medium-Term**

- Offline synchronization.
- Voice-first interaction.
- Smart notifications.
- Satellite weather overlays.
- Integration with agricultural extension services.

---

**Long-Term**

- Drone image analysis.
- Satellite crop monitoring.
- Yield prediction.

- Pest forecasting.
- Smart irrigation recommendations.
- Fertilizer optimization.
- Climate resilience insights.
- Carbon footprint reporting.

---

**Research Opportunities**

- Federated learning for privacy-preserving model improvement.
- Explainable computer vision.
- Predictive outbreak modeling.
- Crop-specific foundation models.
- AI-assisted agricultural policy planning.

---

**Vision Statement**

KissanDost AI aims to become a trusted digital companion that helps farmers make informed decisions while enabling governments and researchers to respond proactively to emerging agricultural challenges.

---

**Chapter 29 — Appendices**

---

**Appendix A – Glossary**

| Term | Meaning |
|---|---|
| AI | Artificial Intelligence |
| RAG | Retrieval-Augmented Generation |
| LLM | Large Language Model |
| API | Application Programming Interface |
| JWT | JSON Web Token |

| Term | Meaning |
|---|---|
| RBAC | Role-Based Access Control |
| UI | User Interface |
| UX | User Experience |
| MVP | Minimum Viable Product |

---

**Appendix B – Key Assumptions**

- Reliable internet access is available for cloud AI services.
- Farmers have access to smartphones or shared devices.
- Trusted agricultural guidance is available for indexing.
- Weather APIs provide accurate regional forecasts.

---

**Appendix C – References**

Suggested sources for implementation and validation:

- FAO (Food and Agriculture Organization)
- Pakistan Agricultural Research Council (PARC)
- KP Agriculture Department
- Open-Meteo documentation
- PostgreSQL documentation
- FastAPI documentation
- Google Gemini documentation

---

**Appendix D – Abbreviations**

| Abbreviation | Full Form |
|---|---|
| CV | Computer Vision |
| NLP | Natural Language Processing |

| Abbreviation | Full Form |
|---|---|
| REST | Representational State Transfer |
| JSON | JavaScript Object Notation |
| SQL | Structured Query Language |

---

**Appendix E – Success Metrics Summary**

| Category | Target |
|---|---|
| AI recommendation time | <10 sec |
| Scan success rate | >95% |
| User satisfaction | >85% |
| System availability | >99% |
| Community participation | >60% (opt-in) |
| Pilot farmers | 100+ |

🎉 **PRD Complete**

This PRD now covers:

- **29 structured chapters**
- Product strategy and vision
- Market analysis
- Functional and non-functional requirements
- UX flows and wireframes
- AI, RAG, and decision engine architecture
- System architecture and database design
- API specifications
- Security and privacy
- Deployment and pilot planning

- Testing, roadmap, and risk analysis
- Demo script and judge preparation
- Future roadmap and appendices